UNIT 1

Introduction to Software Engineering, Software Process Models

Software Engineering Fundamentals

Introduction to Software Engineering & Software Process Models

1. Introduction to Software Engineering

Software Engineering is the systematic, disciplined, and measurable approach to the development, operation, maintenance, and testing of software.

Objectives of Software Engineering

- Develop high-quality software.
- Complete projects within time and budget.
- Reduce development and maintenance costs.
- Improve software reliability and performance.
- Make software easy to maintain and modify.
- Ensure customer satisfaction.

Software Engineering vs. Programming

Programming	Software Engineering
Focuses mainly on writing code	Covers the complete software life cycle
Usually handles smaller tasks	Handles large and complex projects
Less emphasis on documentation	Strong emphasis on documentation
Testing may be limited	Systematic testing is performed
Individual-oriented	Team and process-oriented

2. Software Engineering Fundamentals

2.1 Nature of Software

Software has several characteristics that distinguish it from hardware:

1. **Software is developed, not manufactured**
   Hardware is manufactured in factories, whereas software is created through development activities.
2. **Software does not wear out physically**
   Software does not deteriorate like hardware. However, changes and modifications can introduce defects.
3. **Software is highly complex**
   Large software systems may contain millions of lines of code and numerous interacting components.
4. **Software is easy to modify**
   Changes can be made to software, but frequent changes may increase complexity and maintenance problems.
5. **Software is intangible**
   Unlike hardware, software cannot be physically touched.

Common Types of Software

- System software
- Application software
- Embedded software
- Web applications
- Mobile applications
- Artificial intelligence software
- Real-time software

3. Software Engineering Principles

Important principles include:

1. Modularity

Divide a complex system into smaller, manageable modules.

2. Abstraction

Focus on essential characteristics while hiding unnecessary implementation details.

3. Encapsulation

Keep data and operations together and protect internal implementation details.

4. Separation of Concerns

Divide the system into independent concerns so that each can be handled separately.

5. Reusability

Design components that can be reused in other applications.

6. Maintainability

Software should be easy to understand, modify, and maintain.

7. Simplicity

Avoid unnecessary complexity in design and implementation.

8. Quality Assurance

Quality should be considered throughout the software development process rather than only at the end.

4. The Software Process

A **software process** is a structured set of activities used to develop software.

A generic software process normally includes:

Communication → Planning → Modeling → Construction → Deployment

1. Communication

Understanding customer requirements and expectations.

2. Planning

Estimating:

- Cost
- Schedule
- Resources
- Risks

3. Modeling

Creating:

- Requirements models
- Design models
- Architecture
- Data models

4. Construction

Includes:

- Coding
- Testing

5. Deployment

The software is delivered to customers, evaluated,and maintained.

5. Software Myths

Software myths are incorrect beliefs about software development.

Customer Myths

- "A general statement of objectives is enough to start coding."
- "Changes can easily be made at any time."

Management Myths

- "Our existing standards are sufficient."
- "Adding more programmers will automatically finish the project faster."
- "If the project is late, adding more people will solve the problem."

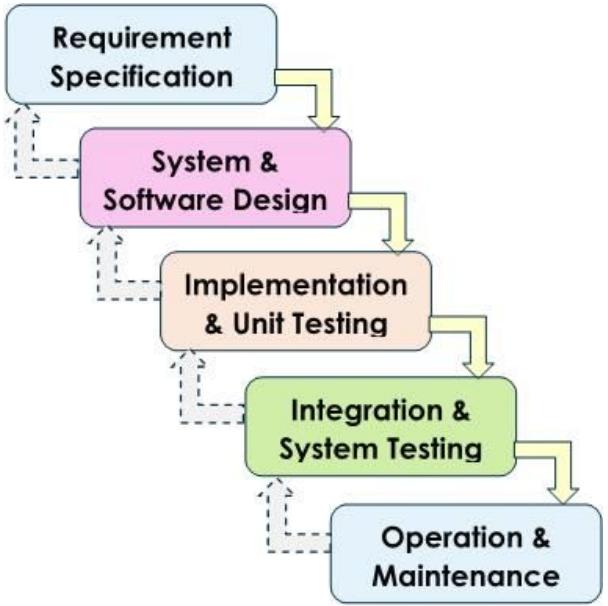
Developer Myths

- "Once the program works, the job is finished."
- "Quality can be achieved only through testing."
- "Working code is the only important deliverable."

Reality: Software development requires systematic requirements analysis, planning, design, coding, testing, deployment, and maintenance.

Software Process Models

A **software process model** is a structured approach that defines **how software is planned, developed, tested, delivered, and maintained**.



The Waterfall Model

1. A Generic Process Model

A **generic process model** provides a common framework for developing software. Most software projects follow five major activities:

1. Communication

- Understand customer requirements.
- Identify the problem and project objectives.
- Discuss requirements with stakeholders.

2. Planning

- Estimate cost, time, and resources.
- Prepare the project schedule.
- Identify risks.

3. Modeling

- Analyze requirements.
- Design the software architecture, database, interfaces, etc.

4. Construction

- Write the program/code.
- Perform testing to identify and remove defects.

5. Deployment

- Deliver the software to users.
- Collect feedback.
- Provide maintenance and updates.

Simple flow:

Communication → Planning → Modeling → Construction → Deployment

Prescriptive Process Models

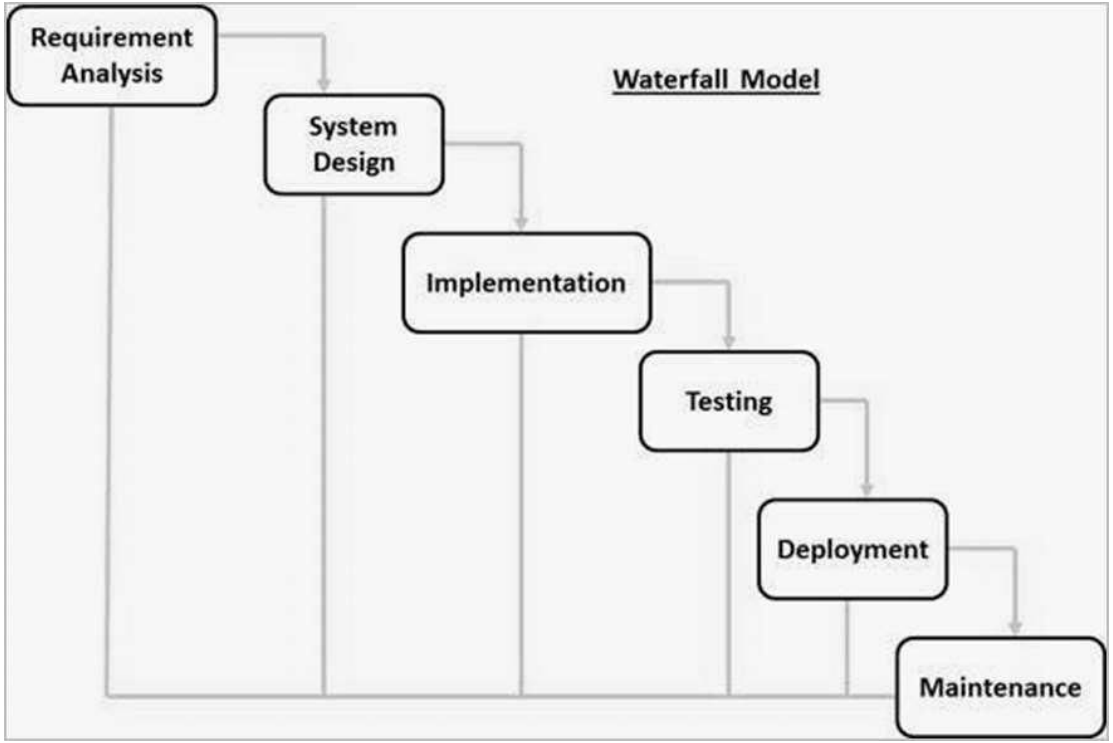
Prescriptive process models define a specific and structured way of performing software development activities.

Major prescriptive models include:

1. Waterfall Model
2. Incremental Process Model
3. RAD Model
4. Evolutionary Process Model
5. Unified Process
6. Concurrent Development Model

Waterfall Model

The **Waterfall Model** is a sequential software development model in which development proceeds through a series of well-defined phases.



Phases

Requirements → Design → Implementation → Testing → Deployment → Maintenance

1. Requirements Analysis

All software requirements are collected and documented.

2. Design

The architecture, database, user interface, and technical design are prepared.

3. Implementation

Developers write the actual source code.

4. Testing

The completed software is tested for errors and defects.

5. Deployment

The software is delivered to the customer.

6. Maintenance

Errors are fixed and necessary changes are made after deployment.

Advantages

- Simple and easy to understand.
- Well-defined phases.
- Easy project management.
- Extensive documentation.
- Suitable when requirements are stable.

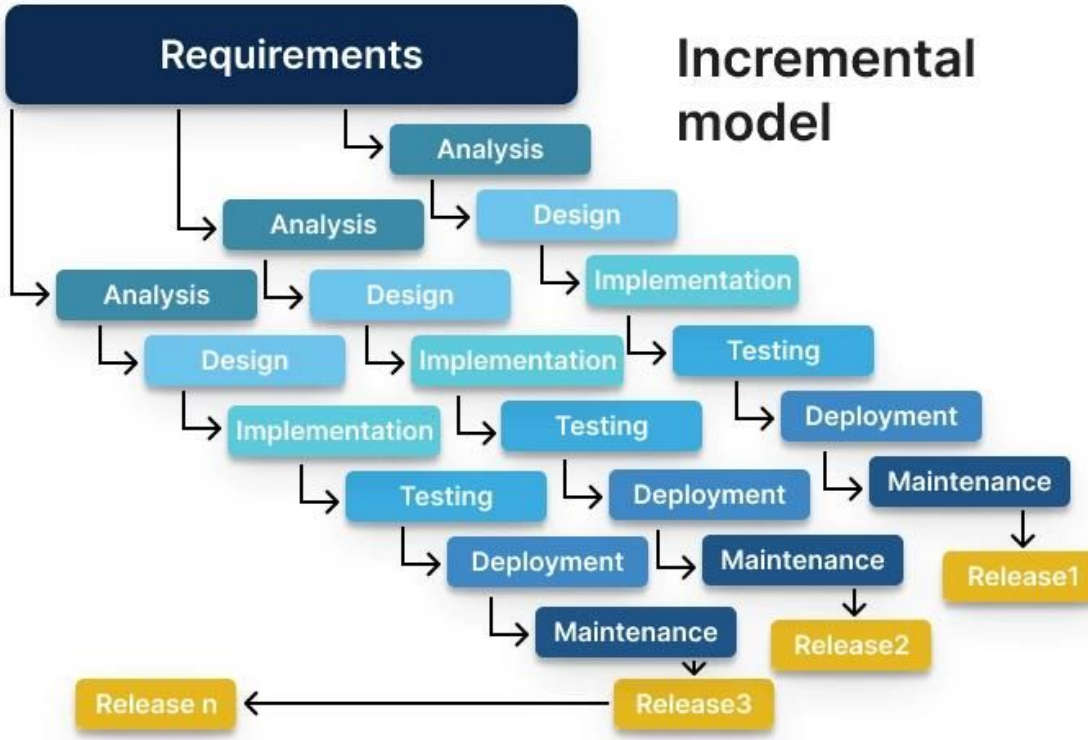
Disadvantages

- Difficult to accommodate changing requirements.
- Testing occurs relatively late.
- Customer feedback is limited during development.
- Working software is available only near the end.

Suitable for

Projects where **requirements are clearly known and unlikely to change**.

Incremental Process Model



Example

Suppose we are developing an **Online Shopping System**.

- **Increment 1:** User registration and login
- **Increment 2:** Product search
- **Increment 3:** Shopping cart
- **Increment 4:** Payment
- **Increment 5:** Order tracking

Each increment provides additional functionality.

Advantages

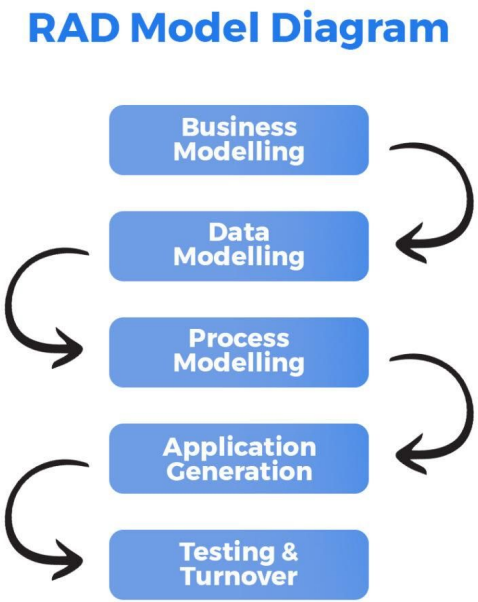
- Working software is delivered early.
- Customer feedback can be incorporated.
- Easier testing and debugging.
- Changes can be accommodated more easily.
- Reduces project risk.

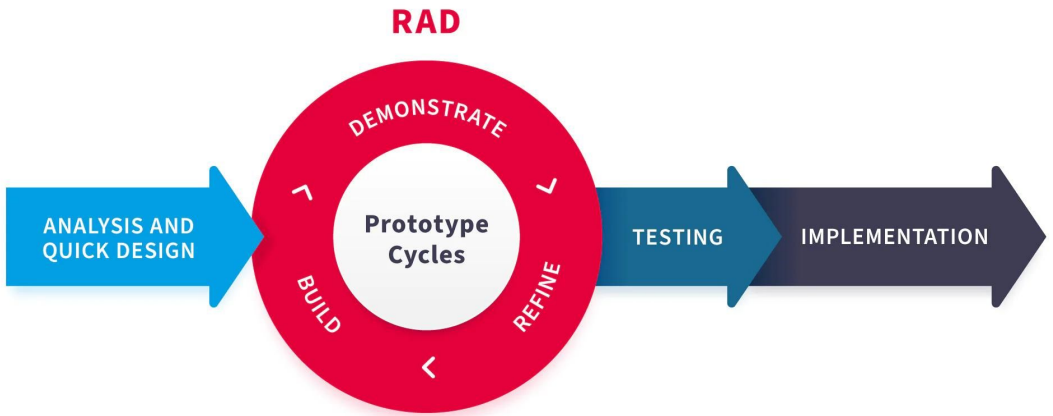
Disadvantages

- Requires good planning.
- Integration can become complex.
- Overall architecture must be planned carefully.
- Not ideal when the system cannot be divided into meaningful increments.

RAD Model RAD = Rapid Application Development

The RAD model focuses on **rapid development and quick delivery** using reusable components, prototyping, and intensive user involvement.





Major Phases

1. **Business Modeling**
2. **Data Modeling**
3. **Process Modeling**
4. **Application Generation**
5. **Testing and Turnover**

Key Features

- Short development cycle.
- Extensive use of reusable components.
- Frequent customer involvement.
- Prototyping.
- Suitable for GUI and database-oriented applications.

Advantages

- Faster development.
- Early delivery.
- Frequent customer feedback.
- Reuse of existing components.

Disadvantages

- Requires skilled developers.
- Requires continuous customer involvement.
- Not suitable for very large or highly complex systems.
- Requires sufficient resources.

Example

Developing a **college management portal** where existing software components and frameworks can be reused.

Evolutionary Process Model

The **Evolutionary Process Model** develops software through repeated cycles. The system evolves as developers and customers gain a better understanding of requirements.

Two important evolutionary models are:

A. Prototyping Model

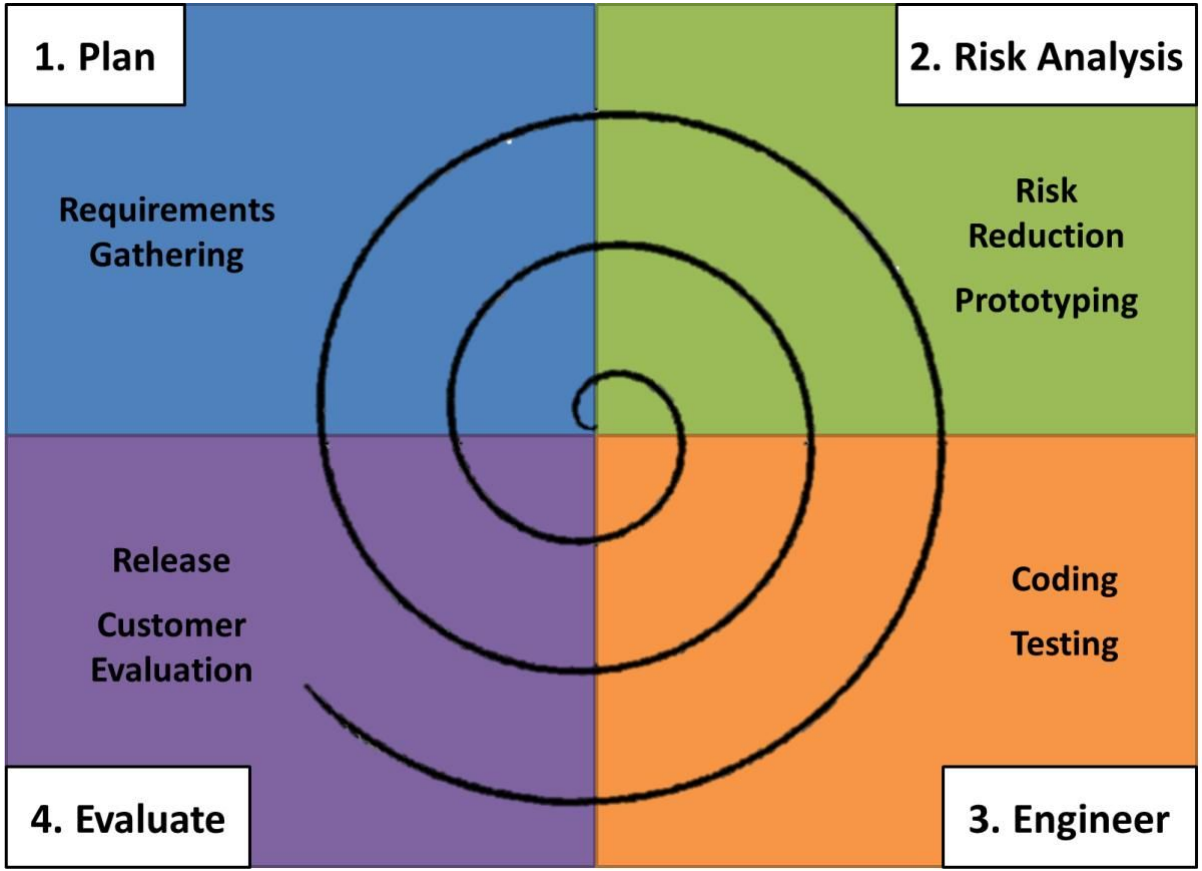
A prototype is created to understand unclear requirements.

Requirements → Prototype → Customer Evaluation → Refinement → Final System

Useful when requirements are not clearly understood.

B. Spiral Model

The Spiral Model combines **iterative development with systematic risk analysis**.


1. Plan
2. Risk Analysis
Requirements Gathering
Risk Reduction
Prototyping
Release
Customer Evaluation
Coding
Testing
4. Evaluate
3. Engineer

Each spiral generally involves:

1. **Planning**
2. **Risk Analysis**
3. **Engineering/Development**
4. **Customer Evaluation**

Advantages

- Strong risk management.
- Suitable for large and complex projects.
- Customer feedback is incorporated.
- Requirements can evolve.

Disadvantages

- More expensive.
- Requires expertise in risk analysis.
- Complex to manage.
- Not suitable for small projects.

Unified Process

The **Unified Process (UP)** is an iterative and incremental software development process.

It divides software development into **four major phases**:

1. Inception

- Define project scope.
- Identify business requirements.
- Determine feasibility.
- Identify major risks.

2. Elaboration

- Analyze detailed requirements.
- Develop architecture.
- Address major technical risks.

3. Construction

- Implement the software.
- Perform testing.
- Develop system functionality.

4. Transition

- Deliver the software to users.
- Perform deployment.
- Provide training and support.

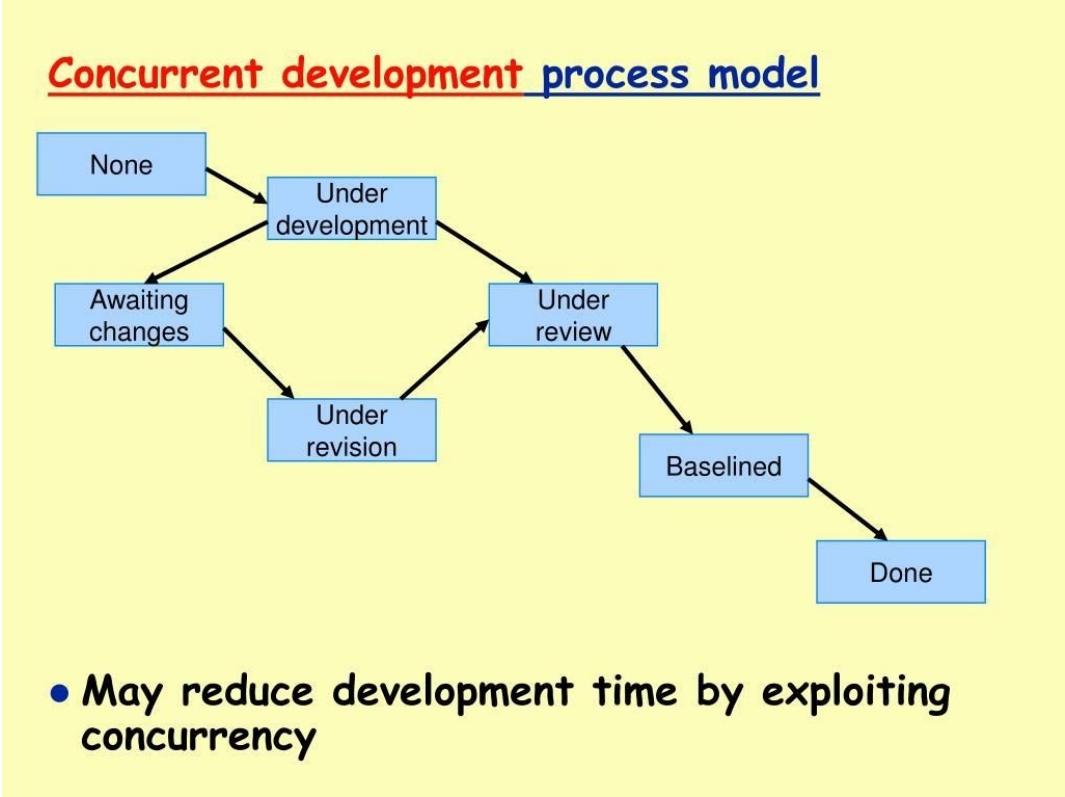
Flow:

Inception → Elaboration → Construction → Transition

Important Characteristics

- Iterative
- Incremental
- Use-case driven
- Architecture-centric
- Risk focused

Concurrent Development Model The **Concurrent Development Model** represents software activities as states that can occur **concurrently**, rather than requiring one activity to completely finish before another begins.



For example, while the design team is working on **design**, the requirements team may still be refining requirements.

An activity can move through states such as:

Under Development → Under Review → Awaiting Changes → Under Revision → Completed

Example

During development:

- Requirements may be **under review**.
- Design may be **under development**.
- Coding may be **under development**.
- Testing may be **awaiting changes**.

Thus, different activities can progress simultaneously.

Advantages

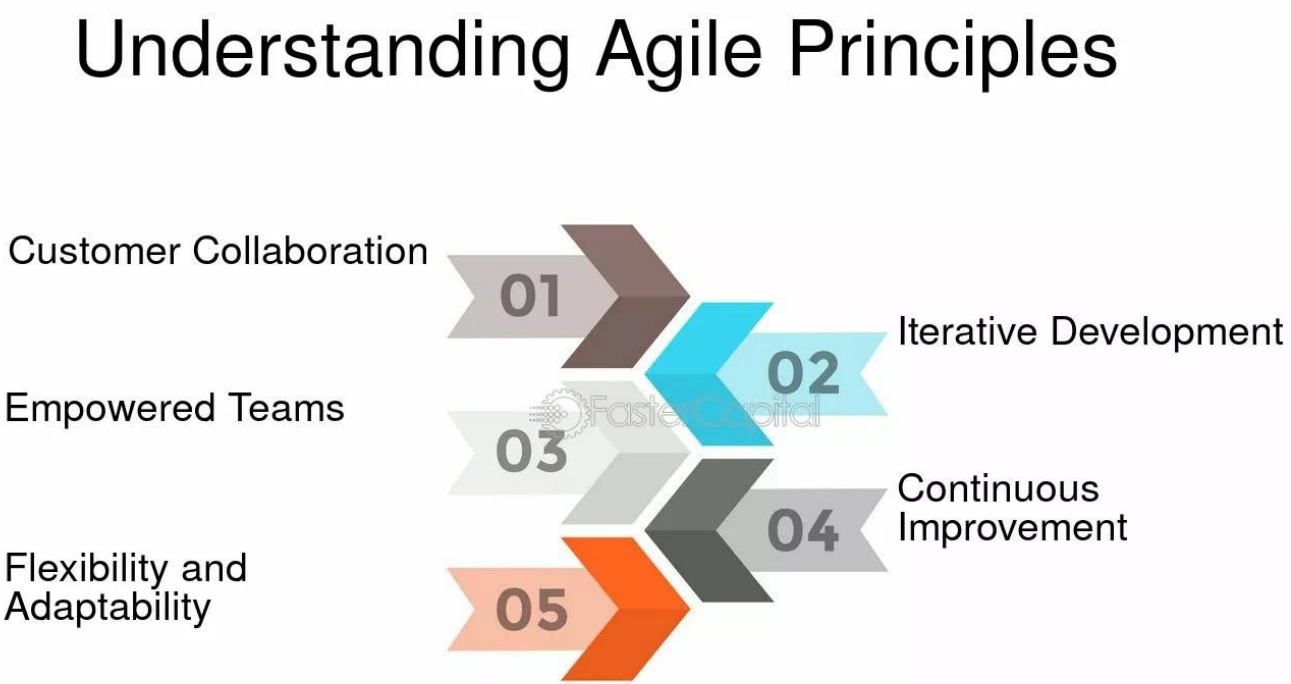
- Supports parallel development.
- Provides faster development.
- Handles changing requirements effectively.
- Provides better visibility of project status.

Disadvantages

- More difficult to manage.
- Requires good coordination.
- Activities may depend on each other.
- Requires experienced project management

1. Agile Software Development

Agile software development is an iterative and incremental approach in which software is developed in **small releases**, with continuous customer feedback and adaptation to changing requirements.



Key Characteristics of Agile

- **Iterative development** – software is developed through repeated cycles.
- **Incremental delivery** – working software is delivered frequently.
- **Customer collaboration** – customers continuously provide feedback.
- **Responding to change** – requirements can change during development.
- **Continuous testing** – testing is performed throughout development.
- **Working software** is given more importance than excessive documentation.

Agile Manifesto – Four Values

1. **Individuals and interactions** over processes and tools.
2. **Working software** over comprehensive documentation.
3. **Customer collaboration** over contract negotiation.
4. **Responding to change** over following a plan.

Agile Methods

Agile is not a single methodology. Several methods follow Agile principles.

Common Agile Methods

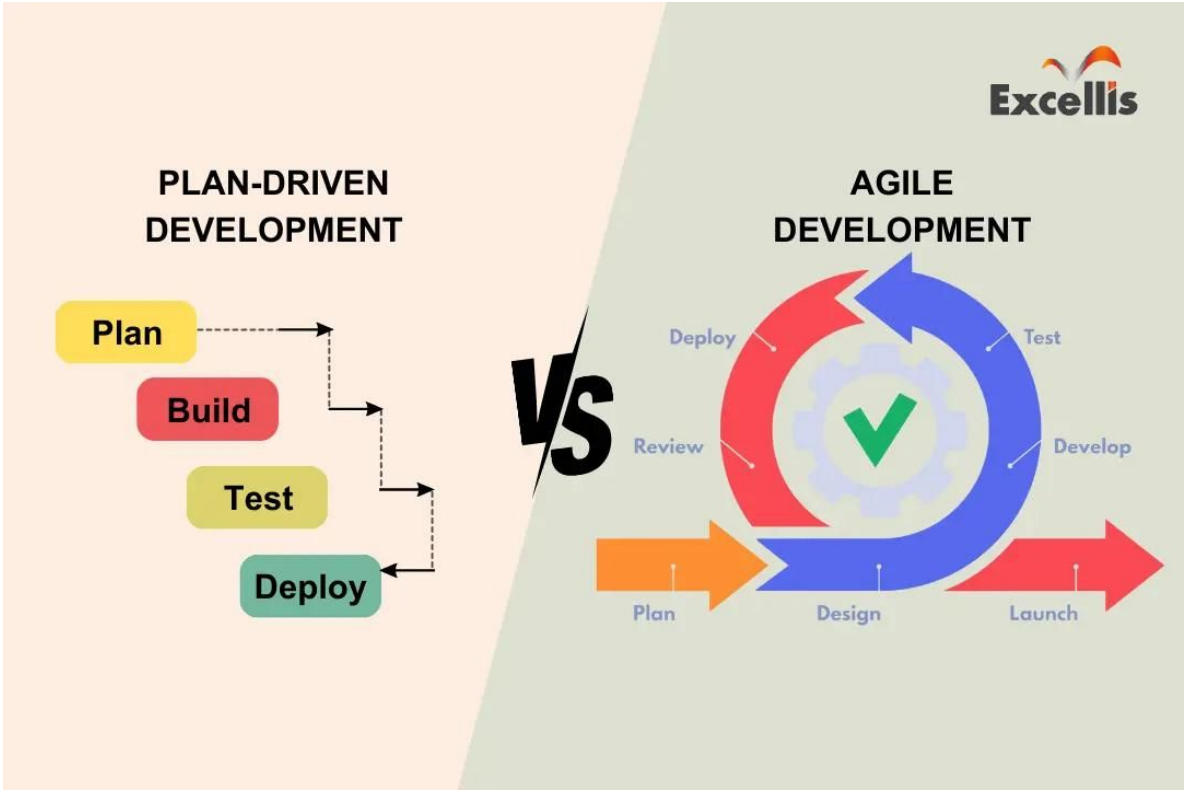
Method	Main Feature
Scrum	Development through short Sprints
Extreme Programming (XP)	Engineering practices and frequent releases
Kanban	Visual workflow and continuous delivery
Crystal	People and communication focused
DSDM	Rapid development and active user involvement
Lean Software Development	Eliminate waste and maximize value

Scrum Example

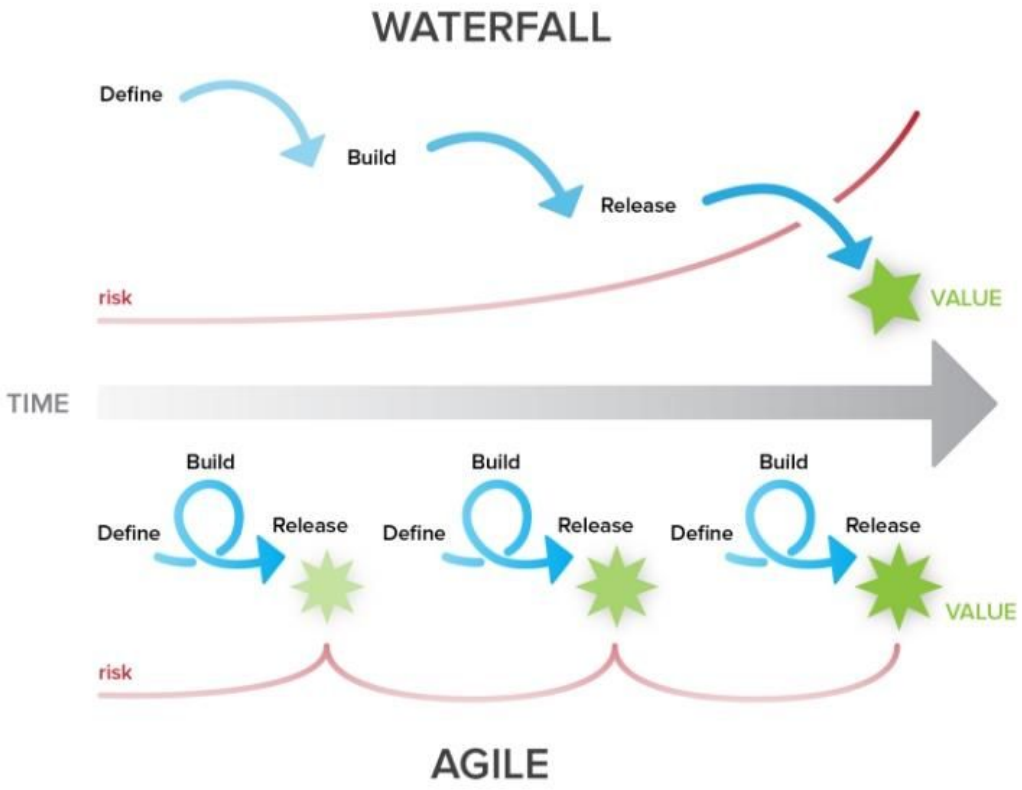
A project can be divided into **2–4 week Sprints**.

Product Backlog → Sprint Planning → Sprint → Daily Scrum → Review → Retrospective → Next Sprint

3. Plan-Driven vs Agile Development



3. Plan-Driven vs Agile Development



Plan-Driven Development

In plan-driven development, **most planning is performed before development begins**.

Example: Waterfall Model

Characteristics:

- Detailed requirements are defined early.
- Extensive documentation.
- Changes are controlled formally.
- Development follows a predefined plan.
- Customer involvement is comparatively lower during development.

Agile Development

Agile development accepts that requirements may change.

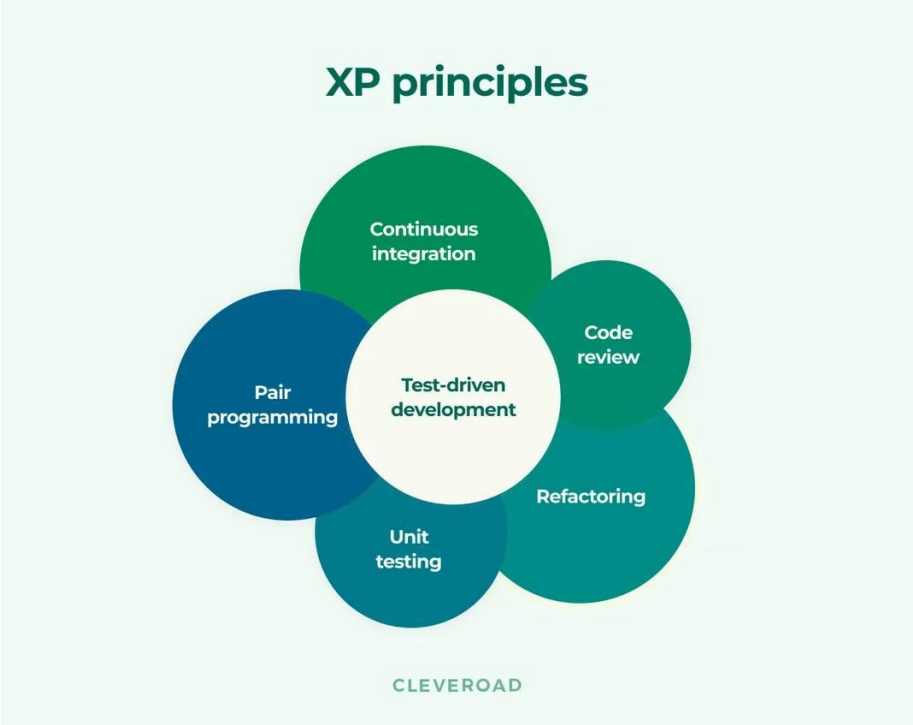
Characteristics:

- Requirements can evolve.
- Frequent working software releases.
- Continuous customer involvement.
- Lightweight documentation.
- Continuous testing and feedback

Factor	Plan-Driven	Agile
Requirements	Mostly fixed	Frequently changing
Planning	Detailed upfront	Continuous
Delivery	Usually at the end	Frequent increments
Customer involvement	Limited/periodic	Continuous
Documentation	Extensive	Usually lightweight
Change	More difficult	Easily accommodated
Testing	Often later phases	Continuous

Extreme Programming (XP)

Extreme Programming (XP) is an Agile software development method that emphasizes **high-quality software, customer involvement, frequent releases, and continuous feedback**.



Important XP Practices

1. Pair Programming

Two programmers work together on the same code.

- **Driver:** Writes the code.
- **Navigator:** Reviews the code and suggests improvements.

They regularly switch roles.

2. Test-Driven Development (TDD)

Tests are written **before or alongside the implementation**.

Basic cycle:

Write Test → Write Code → Run Test → Refactor

3. Continuous Integration

Developers frequently integrate their code into the main system.

4. Small Releases

Working software is released frequently in small increments.

5. Simple Design

Develop only what is currently required and avoid unnecessary complexity.

6. Refactoring

Improve the internal structure of existing code without changing its external behavior.

7. Customer Involvement

The customer continuously provides requirements and feedback.

8. Collective Code Ownership

Any team member can improve or modify the code.

9. Coding Standards

The team follows common coding conventions.

5. Testing in XP

Testing is an **integral part of XP**, rather than an activity performed only after coding.

Main Types

Unit Testing

- Tests individual classes, functions, or modules.
- Developers commonly create automated unit tests.

Acceptance Testing

- Determines whether the software satisfies customer requirements.
- Often based on user stories.

TDD Process

1. Write a failing test
↓
2. Write minimum code to pass the test
↓
3. Run the test
↓
4. Refactor the code
↓
5. Repeat

Benefits

- Detects defects early.
- Improves code quality.
- Supports frequent changes.
- Provides confidence during refactoring.
- Reduces regression problems.

Pair Programming

Pair programming is an XP practice where **two developers work together at one workstation or on the same development task**.

Roles

Driver

- Writes the code.
- Concentrates on implementation.

Navigator

- Reviews the code.
- Thinks about logic, errors, and improvements.

The two developers periodically **switch roles**.

Advantages

- Fewer programming errors.
- Continuous code review.
- Knowledge sharing.
- Better design decisions.
- Helps train less-experienced developers.

Disadvantages

- Two developers are assigned to one task.
- May initially feel slower.
- Requires good communication and cooperation.

Agile teams use software tools to manage **requirements, tasks, progress, defects, collaboration, and workflow**.

JIRA

Atlassian's **Jira** is widely used for Agile project and issue management.

Jira official website

Common Uses

- Create and manage **user stories**
- Create tasks and bugs
- Sprint planning
- Assign work to team members
- Track progress
- Manage product backlog
- Generate reports
- Monitor project status

Example

For an **Online Shopping System**:

Epic: Online Shopping

User Story:
"As a customer, I want to add products to my cart so that I can purchase them."

Tasks:

- Create cart database table
- Develop Add-to-Cart API
- Design cart interface
- Perform testing

Kanban

Kanban is an Agile workflow management method that uses a **visual board** to represent work.



Basic Kanban Board

To Do	In Progress	Testing	Done
Login	Cart	Payment	Registration
Search	Checkout	Order	Profile

Important Concepts

Visualize Work:

Represent tasks using cards.

Limit Work in Progress (WIP):

Avoid having too many tasks in progress.

Manage Flow:

Move tasks smoothly through the workflow.

Continuous Improvement:

Regularly improve the development process.

Kanban vs Scrum

Kanban	Scrum
Continuous flow	Fixed-length Sprints
No mandatory Sprint	Uses Sprints
WIP limits are important	Sprint commitment is important
Flexible prioritization	Backlog planned for Sprint
Continuous delivery	Increment delivered each Sprint

9. Case Study 1: Mental Health-Care Information System

Consider an **information system for mental health-care services**.

The system may provide:

- Patient registration
- Appointment management
- Doctor/therapist information
- Patient records
- Treatment plans
- Medication records
- Progress monitoring
- Reports

Why Agile is Suitable

Mental health-care requirements can evolve based on feedback from:

- Doctors
- Therapists
- Patients
- Administrators

Therefore, Agile allows the system to be developed incrementally.

Example Agile Development

Sprint 1: Patient registration
↓
Sprint 2: Appointment management
↓
Sprint 3: Patient records
↓
Sprint 4: Treatment and progress tracking
↓
Sprint 5: Reports and additional features

After every Sprint, users provide feedback.

Important Considerations

Because this is a health-care information system, development must also consider:

- Privacy
- Security
- Access control
- Data accuracy
- Confidentiality
- Regulatory requirements

10. Case Study 2: Wilderness Weather System

A **Wilderness Weather System** is designed to provide weather information for remote wilderness areas.

The system may collect and provide:

- Temperature
- Wind speed
- Wind direction
- Humidity
- Rainfall
- Weather forecasts
- Weather warnings

Why Evolutionary/Agile Development?

Weather requirements can be uncertain, and the system may need to adapt to:

- New weather data sources
- Changing forecasting requirements
- User feedback
- Environmental conditions
- New reporting requirements

Example Development

Initial Version

- Temperature
- Wind
- Basic weather display

↓

Next Version

- Humidity
- Rainfall
- Historical data

↓

Next Version

- Weather forecasting
- Alerts
- Advanced visualization

Agile Approach

The team can develop a basic working system first and then **incrementally improve it based on user feedback and operational experience**.

Important 5 Questions (Assignment 1)

1. **Explain the Generic Process Model with its major framework activities.**
2. **Explain the Waterfall Model with its phases, advantages, and disadvantages.**
3. **Explain the Incremental Process Model and RAD Model.**
4. **Explain the Evolutionary Process Model, Unified Process, and Concurrent Development Model.**
5. **Explain Agile Software Development, including Agile methods, XP practices, Testing in XP, Pair Programming, JIRA, and Kanban.**